One-Dimensional Arrays

CIS111 – Week 7 lecture

Sait Erkovan

Faculty of Computer and Information Sciences
Yeditepe University

v2026-10-29

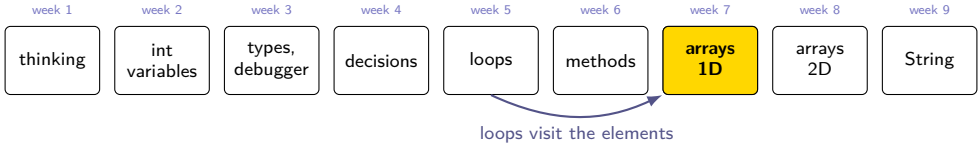
Outline

1. Why arrays?
2. Creating and using an array
3. Common array algorithms
4. Partially filled arrays
5. Arrays and methods
6. Summary

Why arrays?

Week 7: from one value to many values

Where we are



You know (week)	Today
a variable holds one value: `int x` (2–3)	an array holds many: `int[] a`
a counter loop: `for (int i = 0; i < n; i++)` (5)	`i` becomes the index: `a[i]`
sum, count, largest with a loop (5)	the same patterns, on an array
a method with `int` parameters (6)	a method with an `int[]` parameter
the debugger's Variables view (3)	watching all elements at once

Today's problem: five names, or one array?

Five daily highs: what is their average, and on how many days was it warmer?

codeoutput/WithoutArrays.java — Java

```
3         double t0 = 14.0, t1 = 16.5;
4         double t2 = 13.0, t3 = 18.5;
5         double t4 = 13.0;
6         double sum = t0 + t1 + t2 + t3 + t4;
7         double avg = sum / 5;
8         int warm = 0;
9         if (t0 > avg) { warm++; }
10        if (t1 > avg) { warm++; }
11        if (t2 > avg) { warm++; }
12        if (t3 > avg) { warm++; }
13        if (t4 > avg) { warm++; }
14        System.out.println(avg + " " + warm);
```

Today's problem: five names, or one array?

Five daily highs: what is their average, and on how many days was it warmer?

codeoutput/WithoutArrays.java — Java

```
3         double t0 = 14.0, t1 = 16.5;
4         double t2 = 13.0, t3 = 18.5;
5         double t4 = 13.0;
6         double sum = t0 + t1 + t2 + t3 + t4;
7         double avg = sum / 5;
8         int warm = 0;
9         if (t0 > avg) { warm++; }
10        if (t1 > avg) { warm++; }
11        if (t2 > avg) { warm++; }
12        if (t3 > avg) { warm++; }
13        if (t4 > avg) { warm++; }
14        System.out.println(avg + " " + warm);
```

Output — codeoutput/WithoutArrays.txt

```
15.0 2
```

Q. And for 365 days?

Today's problem: five names, or one array?

Five daily highs: what is their average, and on how many days was it warmer?

codeoutput/WithoutArrays.java — Java

```
3         double t0 = 14.0, t1 = 16.5;
4         double t2 = 13.0, t3 = 18.5;
5         double t4 = 13.0;
6         double sum = t0 + t1 + t2 + t3 + t4;
7         double avg = sum / 5;
8         int warm = 0;
9         if (t0 > avg) { warm++; }
10        if (t1 > avg) { warm++; }
11        if (t2 > avg) { warm++; }
12        if (t3 > avg) { warm++; }
13        if (t4 > avg) { warm++; }
14        System.out.println(avg + " " + warm);
```

Output — codeoutput/WithoutArrays.txt

```
15.0 2
```

Q. And for 365 days?

codeoutput/WithAnArray.java — Java

```
3         double[] t = {14.0, 16.5, 13.0,
4                       18.5, 13.0};
5         double sum = 0;
6         for (int i = 0; i < t.length; i++) {
7             sum = sum + t[i];
8         }
9         double avg = sum / t.length;
10        int warm = 0;
11        for (int i = 0; i < t.length; i++) {
12            if (t[i] > avg) { warm++; }
13        }
14        System.out.println(avg + " " + warm);
```

Today's problem: five names, or one array?

Five daily highs: what is their average, and on how many days was it warmer?

codeoutput/WithoutArrays.java — Java

```
3         double t0 = 14.0, t1 = 16.5;
4         double t2 = 13.0, t3 = 18.5;
5         double t4 = 13.0;
6         double sum = t0 + t1 + t2 + t3 + t4;
7         double avg = sum / 5;
8         int warm = 0;
9         if (t0 > avg) { warm++; }
10        if (t1 > avg) { warm++; }
11        if (t2 > avg) { warm++; }
12        if (t3 > avg) { warm++; }
13        if (t4 > avg) { warm++; }
14        System.out.println(avg + " " + warm);
```

Output — codeoutput/WithoutArrays.txt

```
15.0 2
```

codeoutput/WithAnArray.java — Java

```
3         double[] t = {14.0, 16.5, 13.0,
4                       18.5, 13.0};
5         double sum = 0;
6         for (int i = 0; i < t.length; i++) {
7             sum = sum + t[i];
8         }
9         double avg = sum / t.length;
10        int warm = 0;
11        for (int i = 0; i < t.length; i++) {
12            if (t[i] > avg) { warm++; }
13        }
14        System.out.println(avg + " " + warm);
```

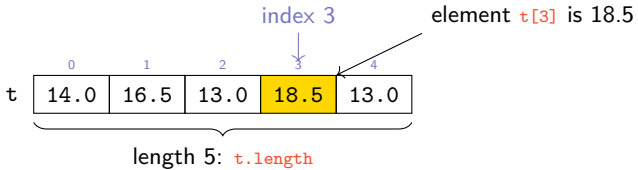
Output — codeoutput/WithAnArray.txt

```
15.0 2
```

Q. And for 365 days? Left: 365 names and 365 `if`s. Right: one name `t`; only the list in lines 3–4 grows.

One name for many values: the parts of an array

```
double[] t = {14.0, 16.5, 13.0, 18.5, 13.0};   // one name, five values
System.out.println(t[3]);                        // the element at index 3
```



Term	Meaning	In Java
element	one stored value	`t[3]`
index	its position: $0, 1, \ldots, n-1$; the first one is 0	`3`, `i`
length n	how many elements; the last index is $n-1$	`t.length`
element type	the type of *every* element	`double`

Creating and using an array

Create it, visit it with a loop, avoid the common errors

Declaring and creating: two steps

```
double[] highs;              // 1. a variable that can refer to a double array
highs = new double[5];       // 2. the array itself: 5 elements, all 0.0

double[] lows = new double[5];   // both steps in one line
```

`double[]`	`lows`	=	`new double[5]`
element type and `[]`	name		creates 5 elements; 5 is the length

- The brackets belong to the type: `double[]` is read "double array"
- `int[] a;` alone creates *no* array yet; using `a[0]` does not compile

A first array

codeoutput/FirstArray.java — Java

```java
public class FirstArray {
    public static void main(String[] args) {
        int[] scores = new int[4];       // 4 elements, all 0
        scores[0] = 85;
        scores[1] = 72;
        scores[3] = 90;                  // scores[2] stays 0
        for (int i = 0; i < scores.length; i++) {
            System.out.println(i + ": " + scores[i]);
        }
        String[] days = {"Mon", "Tue", "Wed"};   // a list
        System.out.println(days.length + " days");
        System.out.println(days[days.length - 1]);
    }
}
```

Q. What does it print? Look at line 6.

A first array

codeoutput/FirstArray.java — Java

```java
public class FirstArray {
    public static void main(String[] args) {
        int[] scores = new int[4];       // 4 elements, all 0
        scores[0] = 85;
        scores[1] = 72;
        scores[3] = 90;                  // scores[2] stays 0
        for (int i = 0; i < scores.length; i++) {
            System.out.println(i + ": " + scores[i]);
        }
        String[] days = {"Mon", "Tue", "Wed"};   // a list
        System.out.println(days.length + " days");
        System.out.println(days[days.length - 1]);
    }
}
```

Output
codeoutput/FirstArray.txt

```
0: 85
1: 72
2: 0
3: 90
3 days
Wed
```

Q. What does it print? Look at line 6. Nobody wrote `scores[2]`: it keeps its start value 0.

- A new array is never "empty": numbers start as 0 (0.0), `boolean` as `false`, `String` as `null`
- A list `{...}` sets the values and the length. The last element is always `a[a.length - 1]`.

Quick check: create

```
int[] a = new int[5];
```

1. What is `a.length`?
2. What is the last valid index?
3. What is `a[2]` right now?
4. Make the array `{3, 1, 4}` in one line.
5. For that `b`, what is `b[b.length - 1]`?

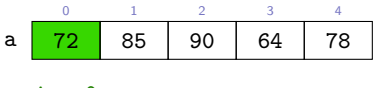
Quick check: create

```
int[] a = new int[5];
```

1. What is `a.length`? 5
2. What is the last valid index? 4
3. What is `a[2]` right now? 0, the start value
4. Make the array `{3, 1, 4}` in one line. `int[] b = {3, 1, 4};`
5. For that `b`, what is `b[b.length - 1]`? 4, the last element

Using the elements: read, write, visit

Write	Meaning
`a[i]`	the element at index `i`: read it
`a[i] = 7;  a[i]++;`	write it, change it, like any variable
`a[i + 1]`, `a[2 * k]`	the index can be any `int` expression
`a[a.length - 1]`	the last element

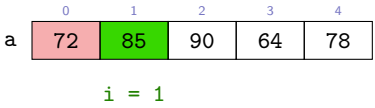



```
for (int i = 0; i < a.length; i++) {      // the counter loop of week 5 ...
    System.out.println(a[i]);             // ... and i is the index
}
```

Start at 0, continue while `i < a.length`. Every algorithm today is this loop with a different body.

Using the elements: read, write, visit

Write	Meaning
`a[i]`	the element at index `i`: read it
`a[i] = 7;  a[i]++;`	write it, change it, like any variable
`a[i + 1]`, `a[2 * k]`	the index can be any `int` expression
`a[a.length - 1]`	the last element

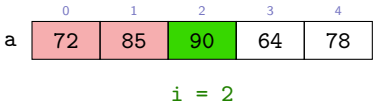



```
for (int i = 0; i < a.length; i++) {      // the counter loop of week 5 ...
    System.out.println(a[i]);             // ... and i is the index
}
```

Start at 0, continue while `i < a.length`. Every algorithm today is this loop with a different body.

Using the elements: read, write, visit

Write	Meaning
`a[i]`	the element at index `i`: read it
`a[i] = 7;  a[i]++;`	write it, change it, like any variable
`a[i + 1]`, `a[2 * k]`	the index can be any `int` expression
`a[a.length - 1]`	the last element

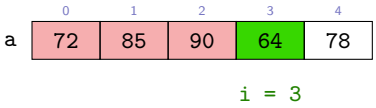



```
for (int i = 0; i < a.length; i++) {      // the counter loop of week 5 ...
    System.out.println(a[i]);             // ... and i is the index
}
```

Start at 0, continue while `i < a.length`. Every algorithm today is this loop with a different body.

Using the elements: read, write, visit

Write	Meaning
`a[i]`	the element at index `i`: read it
`a[i] = 7;  a[i]++;`	write it, change it, like any variable
`a[i + 1]`, `a[2 * k]`	the index can be any `int` expression
`a[a.length - 1]`	the last element

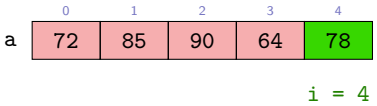



```
for (int i = 0; i < a.length; i++) {      // the counter loop of week 5 ...
    System.out.println(a[i]);             // ... and i is the index
}
```

Start at 0, continue while `i < a.length`. Every algorithm today is this loop with a different body.

Using the elements: read, write, visit

Write	Meaning
`a[i]`	the element at index `i`: read it
`a[i] = 7;  a[i]++;`	write it, change it, like any variable
`a[i + 1]`, `a[2 * k]`	the index can be any `int` expression
`a[a.length - 1]`	the last element

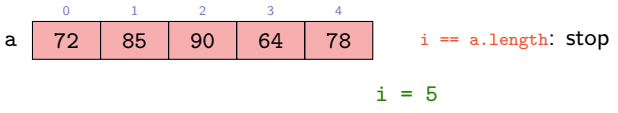



```
for (int i = 0; i < a.length; i++) {      // the counter loop of week 5 ...
    System.out.println(a[i]);             // ... and i is the index
}
```

Start at 0, continue while `i < a.length`. Every algorithm today is this loop with a different body.

Using the elements: read, write, visit

Write	Meaning
`a[i]`	the element at index `i`: read it
`a[i] = 7;  a[i]++;`	write it, change it, like any variable
`a[i + 1]`, `a[2 * k]`	the index can be any `int` expression
`a[a.length - 1]`	the last element




```
for (int i = 0; i < a.length; i++) {      // the counter loop of week 5 ...
    System.out.println(a[i]);             // ... and i is the index
}
```

Start at 0, continue while `i < a.length`. Every algorithm today is this loop with a different body.

The enhanced for loop: for each element

```
for (int s : steps) { ... }      // read: "for each s in steps"
```

`s` takes the values `steps[0]`, `steps[1]`, ... in order, one per turn. There is no index: it is the index loop with `int s = steps[i];` as its first line.

codeoutput/ForEachLoop.java — Java

```
3         int[] steps = {4200, 7800, 6100, 9900};
4         int total = 0;
5         for (int s : steps) {          // for each s in steps
6             total = total + s;
7         }
8         System.out.println("total " + total);
9         for (int s : steps) {
10            s = s * 2;                 // only the copy s
11        }
12        System.out.println(steps[0]);
```

The enhanced for loop: for each element

```
for (int s : steps) { ... }      // read: "for each s in steps"
```

`s` takes the values `steps[0]`, `steps[1]`, ... in order, one per turn. There is no index: it is the index loop with `int s = steps[i];` as its first line.

codeoutput/ForEachLoop.java — Java

```
3         int[] steps = {4200, 7800, 6100, 9900};
4         int total = 0;
5         for (int s : steps) {          // for each s in steps
6             total = total + s;
7         }
8         System.out.println("total " + total);
9         for (int s : steps) {
10            s = s * 2;                 // only the copy s
11        }
12        System.out.println(steps[0]);
```

Output
codeoutput/ForEachLoop.txt

```
total 28000
4200
```

Read only: `s` is a copy, so line 10 does not change the array. To change elements, or to know the index, use the index loop.

Find the two bugs: index out of bounds

Q. The sum should be 60. Find the two bugs before the program runs.

codeoutput/OffByOne.java — Java

```
3           int[] marks = {10, 20, 30};
4           int sum = 0;
5           for (int i = 1; i <= marks.length; i++) {
6               sum = sum + marks[i];
7           }
8           System.out.println(sum);
```

Find the two bugs: index out of bounds

Q. The sum should be 60. Find the two bugs before the program runs.

codeoutput/OffByOne.java — Java

```
3           int[] marks = {10, 20, 30};
4           int sum = 0;
5           for (int i = 1; i <= marks.length; i++) {
6               sum = sum + marks[i];
7           }
8           System.out.println(sum);
```

Output — codeoutput/OffByOne.txt

```
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: Index 3 out of bounds for length 3
        at OffByOne.main(OffByOne.java:6)
```

Find the two bugs: index out of bounds

Q. The sum should be 60. Find the two bugs before the program runs.

codeoutput/OffByOne.java — Java

```
3           int[] marks = {10, 20, 30};
4           int sum = 0;
5           for (int i = 1; i <= marks.length; i++) {
6               sum = sum + marks[i];
7           }
8           System.out.println(sum);
```

Output — codeoutput/OffByOne.txt

```
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: Index 3 out of bounds for length 3
        at OffByOne.main(OffByOne.java:6)
```

1. `i = 1`: `marks[0]` is never added. Start at 0.
2. `i <= marks.length`: `i` reaches 3, but the last index is 2. Use `<`.

Java checks every index. A wrong one stops the program with an error message (an *exception*, week 11); it names the index, the length and the line.

Two errors the compiler finds

Output codeoutput/NotInitialized.txt

```
NotInitialized.java:4: error: variable marks might not have been initialized
                marks[0] = 10;
                ^
1 error
```

`int[] marks;` is only the variable. Create the array: `int[] marks = new int[3];`

Output codeoutput/LengthParens.txt

```
LengthParens.java:4: error: cannot find symbol
                System.out.println(marks.length());
                                           ^
  symbol:   method length()
  location: variable marks of type int[]
1 error
```

The length of an array has no parentheses: `marks.length`. A `String` has `s.length()` (week 9). The compiler names the file and the line; the caret `^` marks the place.

Printing an array

codeoutput/PrintArray.java **Java**

```java
import java.util.Arrays;
public class PrintArray {
    public static void main(String[] args) {
        int[] marks = {10, 20, 30};
        System.out.println(marks);      // not the elements!
        for (int i = 0; i < marks.length; i++) {
            System.out.print(marks[i] + ", ");
        }
        System.out.println();           // one comma too many
        for (int i = 0; i < marks.length; i++) {
            if (i > 0) {
                System.out.print(", "); // not before [0]
            }
            System.out.print(marks[i]);
        }
        System.out.println();
        System.out.println(Arrays.toString(marks));
    }
}
```

Printing an array

codeoutput/PrintArray.java — Java

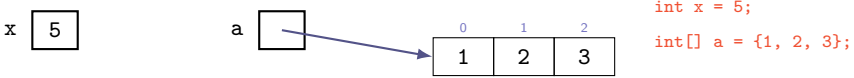
```
1  import java.util.Arrays;
2  public class PrintArray {
3      public static void main(String[] args) {
4          int[] marks = {10, 20, 30};
5          System.out.println(marks);      // not the elements!
6          for (int i = 0; i < marks.length; i++) {
7              System.out.print(marks[i] + ", ");
8          }
9          System.out.println();           // one comma too many
10         for (int i = 0; i < marks.length; i++) {
11             if (i > 0) {
12                 System.out.print(", "); // not before [0]
13             }
14             System.out.print(marks[i]);
15         }
16         System.out.println();
17         System.out.println(Arrays.toString(marks));
18     }
19 }
```

Output
codeoutput/PrintArray.txt

```
[I@2a139a55
10, 20, 30,
10, 20, 30
[10, 20, 30]
```

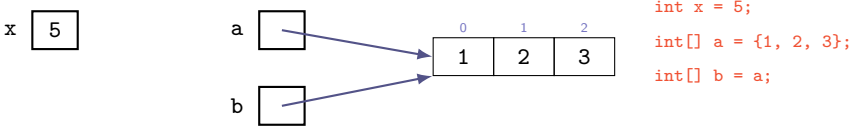
`[I@...` is where the array is. `Arrays.toString` puts the commas for us.

An array variable holds a reference



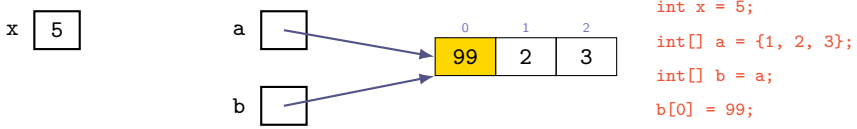
- An `int` variable holds its number. An array variable holds an arrow to its array: a reference.

An array variable holds a reference



- An `int` variable holds its number. An array variable holds an arrow to its array: a reference.
- `b = a` copies the arrow, not the array: `a` and `b` are two names for one array, aliases.

An array variable holds a reference



- An `int` variable holds its number. An array variable holds an arrow to its array: a reference.
- `b = a` copies the arrow, not the array: `a` and `b` are two names for one array, aliases.
- `b[0] = 99` changes that one array, so `a[0]` is 99, too. (After `int y = x;` changing `y` never changes `x`.)

Copying an array: make a new one

codeoutput/CopyArray.java **Java**

```java
import java.util.Arrays;
public class CopyArray {
    public static void main(String[] args) {
        int[] a = {1, 2, 3};
        int[] alias = a;                    // no copy
        int[] c = new int[a.length];        // a copy by hand
        for (int i = 0; i < a.length; i++) {
            c[i] = a[i];
        }
        int[] d = Arrays.copyOf(a, a.length); // shorter
        a[0] = 99;
        System.out.println(Arrays.toString(alias));
        System.out.println(Arrays.toString(c));
        System.out.println(Arrays.toString(d));
        System.out.println(a == alias);
        System.out.println(c == d);
        System.out.println(Arrays.equals(c, d));
    }
}
```

Copying an array: make a new one

codeoutput/CopyArray.java — Java

```java
import java.util.Arrays;
public class CopyArray {
    public static void main(String[] args) {
        int[] a = {1, 2, 3};
        int[] alias = a;                    // no copy
        int[] c = new int[a.length];        // a copy by hand
        for (int i = 0; i < a.length; i++) {
            c[i] = a[i];
        }
        int[] d = Arrays.copyOf(a, a.length); // shorter
        a[0] = 99;
        System.out.println(Arrays.toString(alias));
        System.out.println(Arrays.toString(c));
        System.out.println(Arrays.toString(d));
        System.out.println(a == alias);
        System.out.println(c == d);
        System.out.println(Arrays.equals(c, d));
    }
}
```

Output
codeoutput/CopyArray.txt

```
[99, 2, 3]
[1, 2, 3]
[1, 2, 3]
true
false
true
```

`==` asks “the same array?”; `Arrays.equals` asks “the same elements?”.

On paper: draw the arrows

Draw the variables, the arrows and the arrays after each line. Then answer.

```
int[] p = {4, 5};
int[] q = p;
int[] r = java.util.Arrays.copyOf(p, p.length);
q[1] = 0;
r[0] = 9;
```

1. What are `p[0]` and `p[1]`?
2. What is `r[1]`?
3. How many arrays are there?

On paper: draw the arrows

Draw the variables, the arrows and the arrays after each line. Then answer.

```
int[] p = {4, 5};
int[] q = p;
int[] r = java.util.Arrays.copyOf(p, p.length);
q[1] = 0;
r[0] = 9;
```

1. What are `p[0]` and `p[1]`? 4 and 0: `q` is an alias of `p`
2. What is `r[1]`? 5: `r` is a copy, made before `q[1] = 0`
3. How many arrays are there? 2, and three variables

Common array algorithms

Name the pattern first, then write the loop

A catalogue of patterns

Pattern	Question	Start	At element i	Result
Fill	set every element	–	$a[i] \leftarrow f(i)$	filled array
Accumulate	total? average?	$s \leftarrow 0$	$s \leftarrow s + a[i]$	s, s/n
Count	how many match?	$c \leftarrow 0$	if match: $c \leftarrow c + 1$	c
Largest	which is the largest?	$b \leftarrow 0$ (index)	if $a[i] > a[b]$: $b \leftarrow i$	b, $a[b]$
Smallest	which is the smallest?	$b \leftarrow 0$ (index)	if $a[i] < a[b]$: $b \leftarrow i$	b, $a[b]$
Search	where is the first match?	–	stop at the first match	index or -1
Transform	change every element	–	$a[i] \leftarrow g(a[i])$	same array

Largest and smallest start with element 0 and loop from 1. Week 5 had the same patterns for numbers read one by one; now the numbers stay in the array.

Filling and changing every element

codeoutput/FillChange.java — Java

```java
import java.util.Arrays;

public class FillChange {
    public static void main(String[] args) {
        int[] squares = new int[6];
        for (int i = 0; i < squares.length; i++) {
            squares[i] = i * i;                    // from i
        }
        System.out.println(Arrays.toString(squares));

        int[] scores = {72, 95, 58, 88};
        for (int i = 0; i < scores.length; i++) {
            scores[i] = Math.min(scores[i] + 10, 100);
        }
        System.out.println(Arrays.toString(scores));
    }
}
```

Both write `a[i]`, so both need the index loop. Fill: the value comes from `i`. Change: from the old value; `Math.min` keeps it at most 100.

Filling and changing every element

codeoutput/FillChange.java — Java

```java
import java.util.Arrays;

public class FillChange {
    public static void main(String[] args) {
        int[] squares = new int[6];
        for (int i = 0; i < squares.length; i++) {
            squares[i] = i * i;                    // from i
        }
        System.out.println(Arrays.toString(squares));

        int[] scores = {72, 95, 58, 88};
        for (int i = 0; i < scores.length; i++) {
            scores[i] = Math.min(scores[i] + 10, 100);
        }
        System.out.println(Arrays.toString(scores));
    }
}
```

Output
codeoutput/FillChange.txt

```
[0, 1, 4, 9, 16, 25]
[82, 100, 68, 98]
```

Both write `a[i]`, so both need the index loop. Fill: the value comes from `i`. Change: from the old value; `Math.min` keeps it at most 100.

Sum, average and count

codeoutput/SumCount.java **Java**

```java
public class SumCount {
    public static void main(String[] args) {
        int[] scores = {72, 45, 90, 58, 86};

        int sum = 0;                            // accumulate
        int passed = 0;                         // count
        for (int i = 0; i < scores.length; i++) {
            sum = sum + scores[i];
            if (scores[i] >= 60) {
                passed++;
            }
        }
        double avg = (double) sum / scores.length;
        System.out.println("sum " + sum + ", average " + avg);
        System.out.println(passed + " of " + scores.length
                + " passed");
    }
}
```

One loop, two patterns. **Q.** Why `(double)`?

Sum, average and count

codeoutput/SumCount.java — Java

```java
public class SumCount {
    public static void main(String[] args) {
        int[] scores = {72, 45, 90, 58, 86};

        int sum = 0;                          // accumulate
        int passed = 0;                       // count
        for (int i = 0; i < scores.length; i++) {
            sum = sum + scores[i];
            if (scores[i] >= 60) {
                passed++;
            }
        }
        double avg = (double) sum / scores.length;
        System.out.println("sum " + sum + ", average " + avg);
        System.out.println(passed + " of " + scores.length
                + " passed");
    }
}
```

Output codeoutput/SumCount.txt

```
sum 351, average 70.2
3 of 5 passed
```

One loop, two patterns. **Q.** Why `(double)`? Without it `351 / 5` is 70: integer division (week 3).

Largest and smallest

codeoutput/MinMax.java — Java

```java
public class MinMax {
    public static void main(String[] args) {
        String[] days = {"Mon", "Tue", "Wed", "Thu", "Fri"};
        double[] rain = {2.5, 0.0, 11.5, 4.0, 11.5};  // mm

        int wet = 0;                  // index of the best
        int dry = 0;
        for (int i = 1; i < rain.length; i++) {
            if (rain[i] > rain[wet]) {
                wet = i;
            }
            if (rain[i] < rain[dry]) {
                dry = i;
            }
        }
        System.out.println("wettest " + days[wet]);
        System.out.println("driest  " + days[dry]);
    }
}
```

Keep the *index* of the best so far: it gives the value and the day.

Largest and smallest

codeoutput/MinMax.java — Java

```java
public class MinMax {
    public static void main(String[] args) {
        String[] days = {"Mon", "Tue", "Wed", "Thu", "Fri"};
        double[] rain = {2.5, 0.0, 11.5, 4.0, 11.5};  // mm

        int wet = 0;                  // index of the best
        int dry = 0;
        for (int i = 1; i < rain.length; i++) {
            if (rain[i] > rain[wet]) {
                wet = i;
            }
            if (rain[i] < rain[dry]) {
                dry = i;
            }
        }
        System.out.println("wettest " + days[wet]);
        System.out.println("driest  " + days[dry]);
    }
}
```

Output — codeoutput/MinMax.txt

```
wettest Wed
driest  Tue
```

Keep the *index* of the best so far: it gives the value and the day.

On paper: trace the best-so-far loop

`rain = {2.5, 0.0, 11.5, 4.0, 11.5}`; fill one row per step, then click.

`i`	`rain[i]`	`rain[i] > rain[wet]`?	`wet`	`dry`
start	–	–	0	0
1	0.0			
2	11.5			
3	4.0			
4	11.5			

On paper: trace the best-so-far loop

`rain = {2.5, 0.0, 11.5, 4.0, 11.5}`; fill one row per step, then click.

`i`	`rain[i]`	`rain[i] > rain[wet]`?	`wet`	`dry`
start	–	–	0	0
1	0.0	no	0	1
2	11.5	yes	2	1
3	4.0	no	2	1
4	11.5	no, equal	2	1

Wednesday (index 2) wins the tie: `>` keeps the first of equal values. With `>=`, Friday would win.

Linear search

codeoutput/SearchFlag.java — Java

```java
public class SearchFlag {
    public static void main(String[] args) {
        int[] scores = {72, 85, 90, 64, 95};
        int wanted = 64;
        int pos = 0;
        boolean found = false;
        while (pos < scores.length && !found) {
            if (scores[pos] == wanted) {
                found = true;               // stop here
            } else {
                pos++;                      // look further
            }
        }
        if (found) { System.out.println("at index " + pos); }
        else       { System.out.println("not found"); }
    }
}
```

Look at the elements one by one; stop at the end *or* at the first match. As a method (week 6) it would return `pos`, or -1 when not found: -1 is never an index.

Linear search

codeoutput/SearchFlag.java — Java

```java
public class SearchFlag {
    public static void main(String[] args) {
        int[] scores = {72, 85, 90, 64, 95};
        int wanted = 64;
        int pos = 0;
        boolean found = false;
        while (pos < scores.length && !found) {
            if (scores[pos] == wanted) {
                found = true;                // stop here
            } else {
                pos++;                       // look further
            }
        }
        if (found) { System.out.println("at index " + pos); }
        else       { System.out.println("not found"); }
    }
}
```

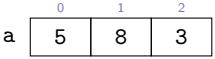
Output
codeoutput/SearchFlag.txt

```
at index 3
```

Look at the elements one by one; stop at the end *or* at the first match. As a method (week 6) it would return `pos`, or -1 when not found: -1 is never an index.

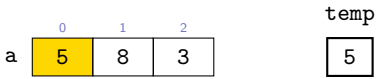
Swapping two elements

Swap `a[0]` and `a[2]`:



Swapping two elements

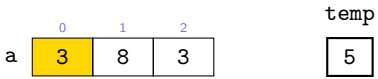
Swap `a[0]` and `a[2]`:



1. `int temp = a[0];` keep a copy of the old `a[0]`

Swapping two elements

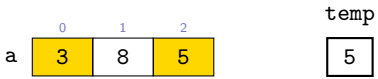
Swap `a[0]` and `a[2]`:



1. `int temp = a[0];` keep a copy of the old `a[0]`
2. `a[0] = a[2];` the old `a[0]` is safe in `temp`

Swapping two elements

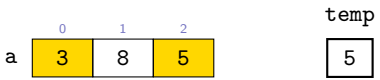
Swap `a[0]` and `a[2]`:



1. `int temp = a[0];` keep a copy of the old `a[0]`
2. `a[0] = a[2];` the old `a[0]` is safe in `temp`
3. `a[2] = temp;`

Swapping two elements

Swap `a[0]` and `a[2]`:



1. `int temp = a[0];` keep a copy of the old `a[0]`
2. `a[0] = a[2];` the old `a[0]` is safe in `temp`
3. `a[2] = temp;`

Without `temp`, step 2 destroys the old `a[0]`: the array becomes `{3, 8, 3}`.

A counting array: the value picks the counter

codeoutput/Histogram.java — Java

```java
/** counts[v]: how often v occurs; the value is the index. */
public class Histogram {
    public static void main(String[] args) {
        int[] scores = {72, 85, 90, 64, 78, 55, 81, 93, 67,
                74, 88, 79, 45, 70, 83, 76, 98, 62, 71, 86};
        int[] counts = new int[11];        // 0-9, ..., 100
        for (int s : scores) {
            counts[s / 10]++;
        }
        for (int b = 4; b <= 9; b++) {
            System.out.print(b * 10 + "s | ");
            for (int k = 0; k < counts[b]; k++) {
                System.out.print("*");
            }
            System.out.println(" " + counts[b]);
        }
    }
}
```

`counts[s / 10]++`: score 72 adds one to `counts[7]`. One pass, no search.

A counting array: the value picks the counter

codeoutput/Histogram.java — Java

```java
/** counts[v]: how often v occurs; the value is the index. */
public class Histogram {
    public static void main(String[] args) {
        int[] scores = {72, 85, 90, 64, 78, 55, 81, 93, 67,
                74, 88, 79, 45, 70, 83, 76, 98, 62, 71, 86};
        int[] counts = new int[11];        // 0-9, ..., 100
        for (int s : scores) {
            counts[s / 10]++;
        }
        for (int b = 4; b <= 9; b++) {
            System.out.print(b * 10 + "s | ");
            for (int k = 0; k < counts[b]; k++) {
                System.out.print("*");
            }
            System.out.println(" " + counts[b]);
        }
    }
}
```

Output
codeoutput/Histogram.txt

```
40s | * 1
50s | * 1
60s | *** 3
70s | ******* 7
80s | ***** 5
90s | *** 3
```

`counts[s / 10]++`: score 72 adds one to `counts[7]`. One pass, no search.

Reverse and palindrome: the mirror index

codeoutput/MirrorIndex.java — Java

```
4     static void reverse(int[] a) {
5         int n = a.length;
6         for (int i = 0; i < n / 2; i++) {
7             int temp = a[i];
8             a[i] = a[n - 1 - i];
9             a[n - 1 - i] = temp;
10        }
11    }
```

codeoutput/MirrorIndex.java — Java

```
13    static boolean isPalindrome(int[] a) {
14        int n = a.length;
15        for (int i = 0; i < n / 2; i++) {
16            if (a[i] != a[n - 1 - i]) {
17                return false;
18            }
19        }
20        return true;
21    }
```

codeoutput/MirrorIndex.java — Java

```
24        int[] a = {1, 2, 3, 4, 5};
25        reverse(a);
26        System.out.println("reversed: " + Arrays.toString(a));
27        int[] p = {3, 1, 4, 1, 3};
28        System.out.println("palindrome? " + isPalindrome(p));
```

Element `i` and its mirror `n - 1 - i`: swap them, or compare them. Half of the array is enough.

Reverse and palindrome: the mirror index

codeoutput/MirrorIndex.java — Java

```
4     static void reverse(int[] a) {
5         int n = a.length;
6         for (int i = 0; i < n / 2; i++) {
7             int temp = a[i];
8             a[i] = a[n - 1 - i];
9             a[n - 1 - i] = temp;
10        }
11    }
```

codeoutput/MirrorIndex.java — Java

```
13    static boolean isPalindrome(int[] a) {
14        int n = a.length;
15        for (int i = 0; i < n / 2; i++) {
16            if (a[i] != a[n - 1 - i]) {
17                return false;
18            }
19        }
20        return true;
21    }
```

codeoutput/MirrorIndex.java — Java

```
24        int[] a = {1, 2, 3, 4, 5};
25        reverse(a);
26        System.out.println("reversed: " + Arrays.toString(a));
27        int[] p = {3, 1, 4, 1, 3};
28        System.out.println("palindrome? " + isPalindrome(p));
```

Output
codeoutput/MirrorIndex.txt

```
reversed: [5, 4, 3, 2, 1]
palindrome? true
```

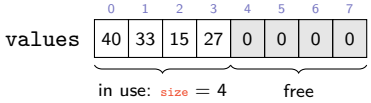
Element `i` and its mirror `n - 1 - i`: swap them, or compare them. Half of the array is enough.

Partially filled arrays

When the number of values is not known in advance

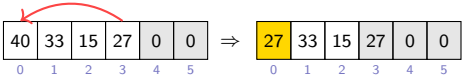
Size, remove, insert

How many values will come? Unknown: make the array big enough and count the cells in use.

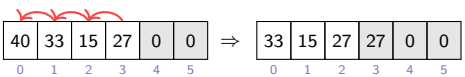


capacity = `values.length` = 8
add at the end: `values[size] = x; size++;`

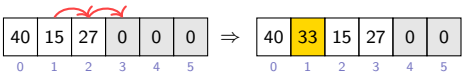
Remove index 0, *any order*: move the last one in



Remove index 0, *keep the order*: shift left



Insert 33 at index 1: shift right, *starting from the end*



Gray cells are outside `size`:
their old values do not matter.
Q. Why must the right shift start from the end?

A partially filled array in Java

codeoutput/PartiallyFilled.java **Java**

```
1  public class PartiallyFilled {
2      static void show(int[] a, int size) {
3          System.out.print(size + " used:");
4          for (int i = 0; i < size; i++) {
5              System.out.print(" " + a[i]);
6          }
7          System.out.println();
8      }
9
10     public static void main(String[] args) {
11         int[] values = new int[8];
12         int size = 0;
13         values[size] = 40; size++;
14         values[size] = 15; size++;
15         values[size] = 27; size++;
16         show(values, size);
```

codeoutput/PartiallyFilled.java **Java**

```
17         for (int i = size; i > 1; i--) {
18             values[i] = values[i - 1];
19         }
20         values[1] = 33; size++;
21         show(values, size);
22         for (int i = 0; i < size - 1; i++) {
23             values[i] = values[i + 1];
24         }
25         size--;
26         show(values, size);
```

`size` travels with the array; `show` loops to `size`, not to `a.length`. Lines 17–20 insert 33 at index 1; lines 22–25 remove index 0 and keep the order.

A partially filled array in Java

codeoutput/PartiallyFilled.java — Java

```
1  public class PartiallyFilled {
2      static void show(int[] a, int size) {
3          System.out.print(size + " used:");
4          for (int i = 0; i < size; i++) {
5              System.out.print(" " + a[i]);
6          }
7          System.out.println();
8      }
9
10     public static void main(String[] args) {
11         int[] values = new int[8];
12         int size = 0;
13         values[size] = 40; size++;
14         values[size] = 15; size++;
15         values[size] = 27; size++;
16         show(values, size);
```

codeoutput/PartiallyFilled.java — Java

```
17         for (int i = size; i > 1; i--) {
18             values[i] = values[i - 1];
19         }
20         values[1] = 33; size++;
21         show(values, size);
22         for (int i = 0; i < size - 1; i++) {
23             values[i] = values[i + 1];
24         }
25         size--;
26         show(values, size);
```

Output — codeoutput/PartiallyFilled.txt

```
3 used: 40 15 27
4 used: 40 33 15 27
3 used: 33 15 27
```

`size` travels with the array; `show` loops to `size`, not to `a.length`. Lines 17–20 insert 33 at index 1; lines 22–25 remove index 0 and keep the order.

Reading input into an array

The count comes first: read it, then make exactly that many elements.

```
int n = in.nextInt();                    // how many values?
double[] t = new double[n];              // exactly n elements
for (int i = 0; i < n; i++) { t[i] = in.nextDouble(); }
```

The count is not known: a big-enough array and `size`, a partially filled array.

codeoutput/ReadScores.java — Java

```
5         int[] scores = new int[100];          // big enough
6         int size = 0;
7         Scanner in = new Scanner(System.in);
8         while (size < scores.length && in.hasNextInt()) {
9             scores[size] = in.nextInt();
10            size++;
11        }
12        for (int i = size - 1; i >= 0; i--) {   // backwards
13            System.out.print(scores[i] + " ");
14        }
15        System.out.println("(" + size + " read)");
```

codeoutput/ReadScores.in

```
72 85
90 64
78
q
```

`Scanner` as in the week 3 lab (more in week 10). The loop stops at the first non-number, `q`.

Reading input into an array

The count comes first: read it, then make exactly that many elements.

```java
int n = in.nextInt();                    // how many values?
double[] t = new double[n];              // exactly n elements
for (int i = 0; i < n; i++) { t[i] = in.nextDouble(); }
```

The count is not known: a big-enough array and `size`, a partially filled array.

codeoutput/ReadScores.java — Java

```java
        int[] scores = new int[100];          // big enough
        int size = 0;
        Scanner in = new Scanner(System.in);
        while (size < scores.length && in.hasNextInt()) {
            scores[size] = in.nextInt();
            size++;
        }
        for (int i = size - 1; i >= 0; i--) {   // backwards
            System.out.print(scores[i] + " ");
        }
        System.out.println("(" + size + " read)");
```

codeoutput/ReadScores.in

```
72 85
90 64
78
q
```

Output
codeoutput/ReadScores.txt

```
78 64 90 85 72 (5 read)
```

`Scanner` as in the week 3 lab (more in week 10). The loop stops at the first non-number, `q`.

Growing an array

codeoutput/GrowArray.java — Java

```java
import java.util.Arrays;

public class GrowArray {
    public static void main(String[] args) {
        int[] values = {40, 15, 27};           // full: length 3
        int size = values.length;
        if (size == values.length) {           // no room left:
            values = Arrays.copyOf(values, 2 * values.length);
        }                                      // a new, longer array
        values[size] = 33;
        size++;
        System.out.println(Arrays.toString(values));
        System.out.println("size " + size + " of " + values.length);
    }
}
```

Growing an array

codeoutput/GrowArray.java — Java

```java
import java.util.Arrays;

public class GrowArray {
    public static void main(String[] args) {
        int[] values = {40, 15, 27};           // full: length 3
        int size = values.length;
        if (size == values.length) {           // no room left:
            values = Arrays.copyOf(values, 2 * values.length);
        }                                      // a new, longer array
        values[size] = 33;
        size++;
        System.out.println(Arrays.toString(values));
        System.out.println("size " + size + " of " + values.length);
    }
}
```

Output — codeoutput/GrowArray.txt

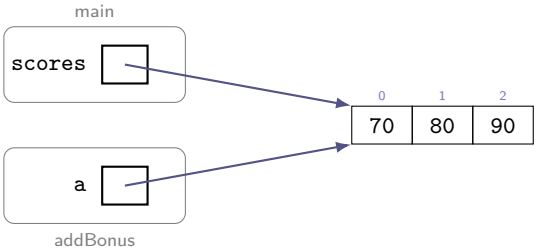
```
[40, 15, 27, 33, 0, 0]
size 4 of 6
```

An array cannot grow: `Arrays.copyOf` makes a longer one and copies.

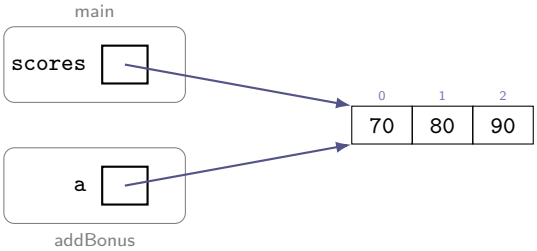
Arrays and methods

The methods of week 6, now with array parameters

Passing an array passes the arrow

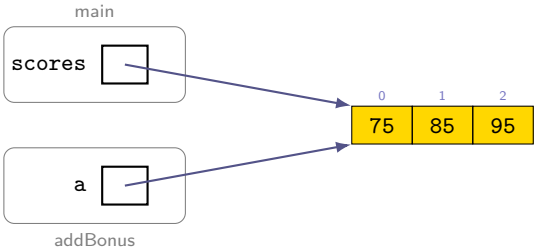


Passing an array passes the arrow



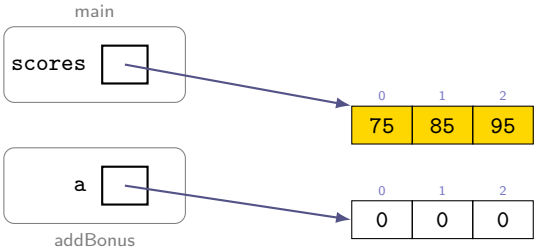
- The parameter `a` gets a copy of the *arrow*: both refer to one array

Passing an array passes the arrow



- The parameter `a` gets a copy of the *arrow*: both refer to one array
- The method changes `a[i]`: the caller sees the change

Passing an array passes the arrow



- The parameter `a` gets a copy of the *arrow*: both refer to one array
- The method changes `a[i]`: the caller sees the change
- The method sets `a` to a new array: only its own arrow moves; the caller does not see it

A method can change an array, or return a new one

codeoutput/PassReference.java — Java

```
3       static void addBonus(int[] a, int bonus) {
4           for (int i = 0; i < a.length; i++) {
5               a[i] = a[i] + bonus;          // the caller's array
6           }
7       }
8       static int[] doubled(int[] a) {
9           int[] result = new int[a.length];
10          for (int i = 0; i < a.length; i++) {
11              result[i] = 2 * a[i];
12          }
13          return result;                    // a new array
14      }
15      public static void main(String[] args) {
16          int[] scores = {70, 80, 90};
17          addBonus(scores, 5);
18          System.out.println(Arrays.toString(scores));
19          int[] big = doubled(scores);
20          System.out.println(Arrays.toString(big));
21      }
```

A method can change an array, or return a new one

codeoutput/PassReference.java — Java

```
3     static void addBonus(int[] a, int bonus) {
4         for (int i = 0; i < a.length; i++) {
5             a[i] = a[i] + bonus;          // the caller's array
6         }
7     }
8     static int[] doubled(int[] a) {
9         int[] result = new int[a.length];
10        for (int i = 0; i < a.length; i++) {
11            result[i] = 2 * a[i];
12        }
13        return result;                    // a new array
14    }
15    public static void main(String[] args) {
16        int[] scores = {70, 80, 90};
17        addBonus(scores, 5);
18        System.out.println(Arrays.toString(scores));
19        int[] big = doubled(scores);
20        System.out.println(Arrays.toString(big));
21    }
```

Output
codeoutput/PassReference.txt

```
[75, 85, 95]
[150, 170, 190]
```

`addBonus` changes the caller's array; `doubled` returns a new one.

Solving a problem: decompose it

Q. The lowest quiz is dropped; what is the average of the rest? Split it (total, lowest), name the patterns (accumulate, best so far), combine, check by hand: $(38 - 4)/4 = 8.5$.

codeoutput/DropLowest.java — Java

```
3         int[] quiz = {8, 4, 10, 7, 9};
4
5         int sum = 0;                    // accumulate
6         int low = quiz[0];              // best so far
7         for (int q : quiz) {
8             sum = sum + q;
9             if (q < low) {
10                low = q;
11            }
12        }
13        int n = quiz.length - 1;
14        double avg = (double) (sum - low) / n;
15        System.out.println(sum + " " + low + " " + avg);
```

Solving a problem: decompose it

Q. The lowest quiz is dropped; what is the average of the rest? Split it (total, lowest), name the patterns (accumulate, best so far), combine, check by hand: $(38 - 4)/4 = 8.5$.

codeoutput/DropLowest.java — Java

```
3         int[] quiz = {8, 4, 10, 7, 9};
4
5         int sum = 0;                    // accumulate
6         int low = quiz[0];              // best so far
7         for (int q : quiz) {
8             sum = sum + q;
9             if (q < low) {
10                low = q;
11            }
12        }
13        int n = quiz.length - 1;
14        double avg = (double) (sum - low) / n;
15        System.out.println(sum + " " + low + " " + avg);
```

Output
codeoutput/DropLowest.txt

```
38 4 8.5
```

In Eclipse: watch an array in the debugger

Action	Windows	macOS
Toggle a breakpoint	Ctrl + Shift + B	⇧ + ⌘ + B
Debug (last launched)	F11	⌘ + F11
Step over / step into	F6 / F5	F6 / F5
Resume to the next breakpoint	F8	F8

1. Double-click the left margin of a line inside the loop: a breakpoint
2. Run » Debug As » Java Application opens the Debug perspective
3. In the Variables view, open the array: one row per element, `[0]`, `[1]`, ...
4. Step with F6: the element that changes is highlighted

macOS: if the F-keys change volume or brightness, also hold fn. Try it on `FillChange`: watch `scores` change one element per step.

Quick check: methods

```
static void renew(int[] a) { a = new int[a.length]; }
static void reset(int[] a) {
    for (int i = 0; i < a.length; i++) { a[i] = 0; }
}
int[] x = {1, 2, 3};   renew(x);   reset(x);
```

1. After `renew(x)`, what is `x[0]`?
2. After `reset(x)`, what is `x[0]`?
3. Write the header of a method that returns the reverse of an `int` array.

Quick check: methods

```
static void renew(int[] a) { a = new int[a.length]; }
static void reset(int[] a) {
    for (int i = 0; i < a.length; i++) { a[i] = 0; }
}
int[] x = {1, 2, 3};   renew(x);   reset(x);
```

1. After `renew(x)`, what is `x[0]`? 1: only the parameter's arrow moved
2. After `reset(x)`, what is `x[0]`? 0: the elements of the caller's array changed
3. Write the header of a method that returns the reverse of an `int` array. `static int[] reversed(int[] a)`

Summary

Cheat sheet

```
int[] a = new int[5];                  // create: 5 zeros
int[] b = {4, 8, 15};                  // create with values
a[0] = 7;   int x = a[2];              // write, read
int n = a.length;                      // no parentheses
int last = a[a.length - 1];            // the last element
int[] c = a;                           // an alias: the same array!
int[] d = Arrays.copyOf(a, a.length); // a copy: a new array
for (int i = 0; i < a.length; i++) {  // index loop: read and write
    a[i] = 2 * a[i];
}
for (int v : a) {                      // enhanced for: read only
    System.out.print(v + " ");
}
static int[] reversed(int[] a) {...}  // array parameter, array result
```

Patterns: fill, accumulate, count, largest and smallest, search, transform, swap. Partially filled arrays: `size`; insert and remove by shifting.

Exercises

1. Store the number of days of the 12 months in an array (February: 28) and print the total.
2. Read up to 50 exam scores until a non-number; print how many are above the average.
3. Write `static int lastIndexOf(int[] a, int x)`: the last index of `x`, or -1. Search from the end.
4. Write `static void rotateLeft(int[] a)`: `{1, 2, 3, 4}` becomes `{2, 3, 4, 1}`. Which element must be saved first?
5. In a partially filled array, remove *every* element equal to `x`, keeping the order.
6. Write `static boolean isIncreasing(int[] a)`: is every element larger than the one before it?

Where arrays come back

Week	Topic	Arrays there
8	2D arrays	a table is an array of rows: `int[][] t`; `t[r][c]`
9	String	a sequence of `char`s: `s.charAt(i)` is like `a[i]`; `s.length()`
10	Input/Output	read a file into a (partially filled) array
11	Exceptions, testing	today's index-out-of-bounds error is an exception; tests compare arrays
12	Recursion	a method that works on a part of an array
13	Command line	`main(String[] args)`: the words typed after the program name

Reading: bjlo2, Chapter 6 (6.1–6.6). Decks of this topic: `array1D-0-concepts` ... `array1D-5-examples`.

References

The Java Tutorials, Arrays [1].

[1] The java tutorials-arrays.

Questions?